Clustering Techniques: K-Means, Hierarchical, and DBSCAN

Dataset: Mall Customer Segmentation Data (Kaggle) Goal: Group customers based on spending behavior.

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
```

2. Load Dataset

```
df = pd.read_csv('Mall_Customers.csv')
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Next steps: Generate code with df New interactive sheet

3. Dataset Information

```
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

4. Select Features for Clustering

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

5. Feature Scaling

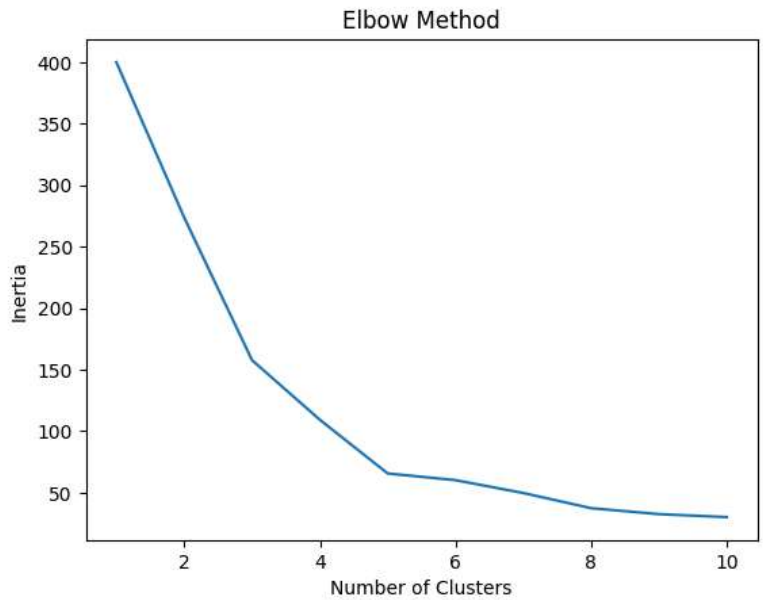
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

6. Elbow Method for Optimal K

```
inertia = []
k_range = range(1,11)

for k in k_range:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(X_scaled)
    inertia.append(model.inertia_)

plt.plot(k_range, inertia)
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```



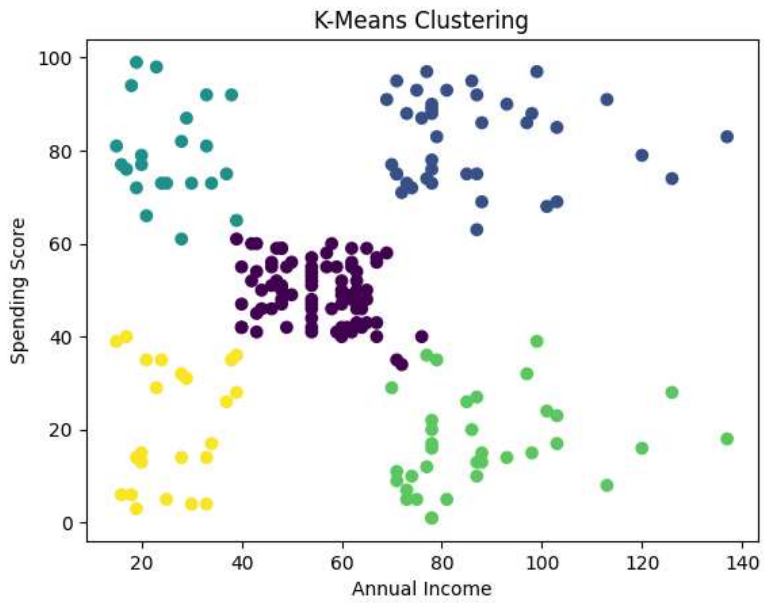
7. K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

df['KMeans_Cluster'] = kmeans_labels
```

8. K-Means Visualization

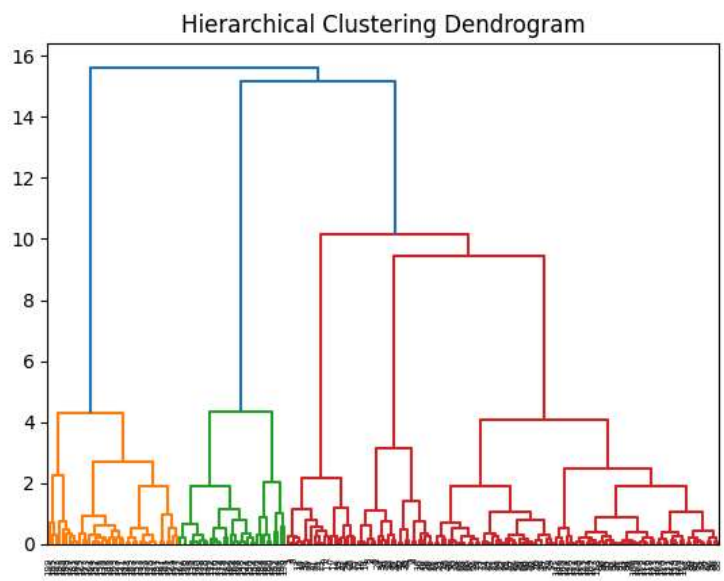
```
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=kmeans_labels)
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.title('K-Means Clustering')
plt.show()
```



9. Hierarchical Clustering (Dendrogram)

```
linked = linkage(X_scaled, method='ward')

plt.figure()
dendrogram(linked)
plt.title('Hierarchical Clustering Dendrogram')
plt.show()
```



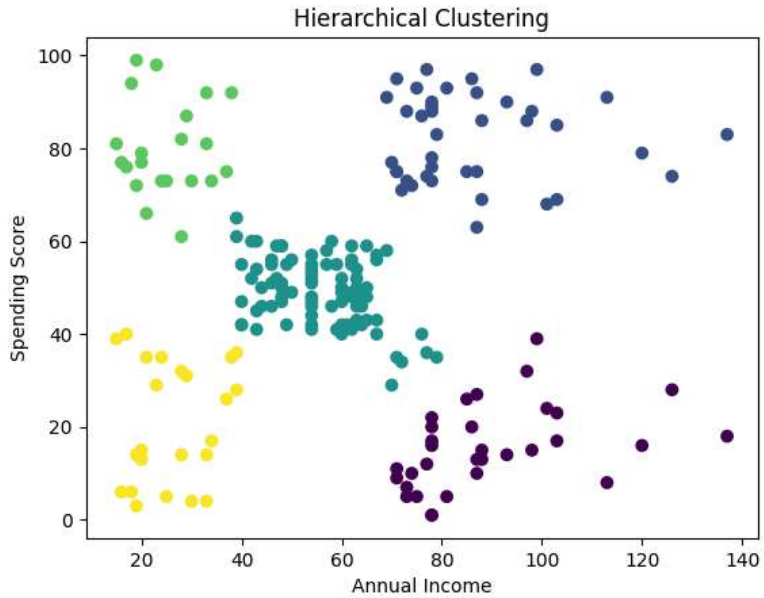
10. Agglomerative Clustering

```
hc = AgglomerativeClustering(n_clusters=5)
hc_labels = hc.fit_predict(X_scaled)

df['Hierarchical_Cluster'] = hc_labels
```

11. Hierarchical Clustering Visualization

```
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=hc_labels)
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.title('Hierarchical Clustering')
plt.show()
```



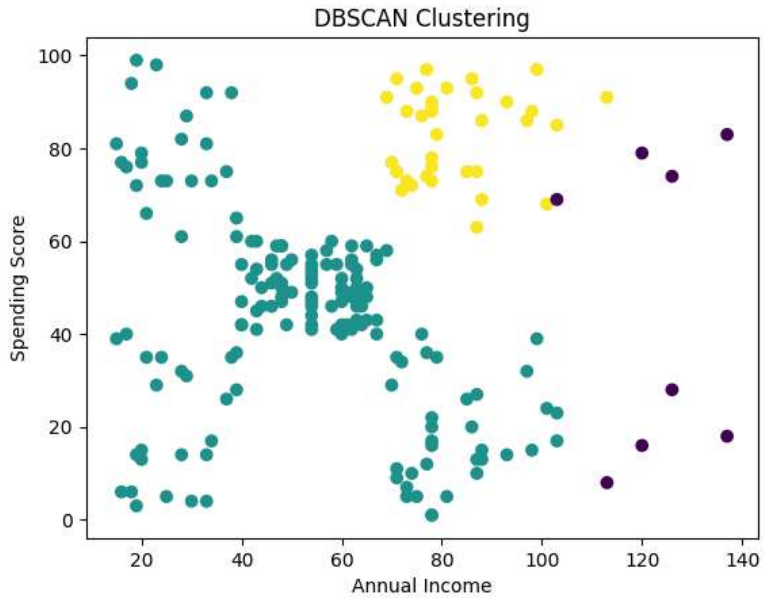
12. DBSCAN Clustering

```
dbscan = DBSCAN(eps=0.5, min_samples=5)
db_labels = dbscan.fit_predict(X_scaled)

df['DBSCAN_Cluster'] = db_labels
```

13. DBSCAN Visualization

```
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=db_labels)
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.title('DBSCAN Clustering')
plt.show()
```



14. Silhouette Score Comparison

```
print('KMeans Silhouette:', silhouette_score(X_scaled, kmeans_labels))
print('Hierarchical Silhouette:', silhouette_score(X_scaled, hc_labels))
```

```
KMeans Silhouette: 0.5546571631111091
Hierarchical Silhouette: 0.5538089226688662
```

Start coding or generate with AI.